

Прохождение внешнего курса на STEPİK.

Третий раздел

Ибрагимов Гаджимурад Шамильвич НКАбд-02-25 1032253512

Операционные системы 2026-05-16

Содержание

Введение	3
Выполнение работы	4
Вывод	19
####5	

Введение

Третья часть посвящена более сложным и интересным темам, таким, как редактор Vim, интерпретатор bash и другие приложения.

Выполнение работы

1. Текстовый редактор vim. На этом занятии мы рассмотрим гораздо более мощный редактор [vim].

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ "Ctrl", затем "x"
- ☐ "q", затем "Enter"
- ☐ "q"
- ☒ ":", затем "q", затем "Enter"
- ☐ "Q"

При перемещении в vim “по словам” есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по “словам” (word), а вторые по “большим словам” (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

Strange_ TEXT is_here. 2=2 YES!

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ В этой строке 12 "слов" (word)
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☒ В этой строке 9 "слов" (word)
- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
- ☒ В этой строке 5 "больших слов" (WORD)
- ☒ Нажимая только на W, нельзя переместить курсор на "."

Следующий шаг

Решить снова

Один из способов:

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✔ Правильно, молодец!

```
:1,$s/Windows/Linux/
```

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Выберите все подходящие ответы из списка

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☒ В режиме выделения можно использовать команды d (удалить) и y (скопировать)

Следующий шаг

Решить снова

2. Скрипты на bash: основы. Надеемся, что вы разобрались, что одну оболочку (например, sh) можно запустить из другой оболочки (например, из bash).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка bash. Вы набираете в ней команды A1, A2, A3, а затем запускаете оболочку sh. В этой оболочке вы набираете команды B1, B2, B3 и запускаете оболочку bash. И, наконец, в этой последней оболочке вы набираете команды C1, C2, C3. Если теперь вы попытаетесь при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Отличное решение!

- ☐ Из наборов B и C
- ☒ Только из набора C
- ☐ Никакие команды появляться не будут
- ☐ Только из набора A
- ☐ Только из набора B

Следующий шаг

Решить снова

Так как создание файла происходило в каталоге bi

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Так точно!

- ☒ `/home/bi/file1.txt`
- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Desktop/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`

Следующий шаг

Решить снова

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте:

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ variable_123
- ☒ _variable
- ☐ 123variable
- ☒ VARiable
- ☒ variable123
- ☒ __variable
- ☐ variab\$\$le

Следующий шаг

Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 39 809 учащихся
Из всех попыток 25% верных

Пропишем следующий код:

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Отлично!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 !/bin/bash
2
3 var1=$1
4 var2=$2
5
6 echo "Arguments are: \ $1=$var1 \ $2=$var2 "
```

3. Скрипты на bash: ветвления и циклы. Предположим, вы пишете скрипт на bash и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
    echo "True"
fi
```

Вы можете вписать вместо “...” (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

```
gkibragimov@fedora:~$ ./test.sh
./test.sh: строка 1: !/bin/bash: Нет такого файла или каталога
3
4
5
gkibragimov@fedora:~$
```

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `-n $1`
- ☐ `$var1 == $var2 && $var1 != $var2`
- ☒ `-e $0`
- ☒ `-z ""`
- ☒ `$var1 == $var2 || $var1 != $var2`
- ☐ `-z " "`

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
    echo "one"
elif [[ $var -lt 3 ]]
then
    echo "two"
elif [[ $var -eq 4 ]]
then
    echo "three"
else
```



```
    echo "four"  
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную **var=3**, а затем запустили еще раз, но уже

```
gkibragimov@fedora:~$ ./test.sh  
four  
gkibragimov@fedora:~$ ./test.sh  
four  
gkibragimov@fedora:~$  
с var=5.
```

Выберите один вариант из списка

☒ Хорошие новости, верно!

- ☒ Сначала four, потом four
- ☐ Сначала four, потом one
- ☐ Сначала two, потом one
- ☐ Сначала two, потом four

Следующий шаг

Решить снова

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students  
1 --> 1 student  
2 --> 2 students  
3 --> 3 students  
4 --> 4 students  
5 и больше --> A lot of students
```

Скрипт:

Напишите программу. Тестируется через stdin → stdout

✓ Всё получилось!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2     echo "$1 student"
3 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
4     echo "$1 students"
5 elif [[ $1 -ge 5 ]]; then
6     echo "A lot of students"
7 else
8     echo "No students"
9 fi
10
```

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
    echo "start"
    if [[ $str > "c" ]]
    then
        continue
    fi
    echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово “start”, а сколько раз слово “finish”?

```
gkibragimov@fedora:~$ ./test.sh
start
finish
start
finish
start
finish
start
finish
start
finish
gkibragimov@fedora:~$
```

Выберите один вариант из списка

✓ Здорово, всё верно.

- ☐ 5 раз "start" и 2 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 5 раз "start" и ни разу "finish"

Следующий шаг

Решить снова

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение “**enter your name:**” и ждать от пользователя ввода имени (используйте read, чтобы про-

читать его). Когда имя введено, то скрипт должен написать **“enter your age:”** и ждать ввода возраста (опять нужен read). Когда возраст введен, скрипт пишет на экран **“, your group is ”**, где определяется на основе возраста по следующим правилам:

- младше либо равно 16: **“child”**,
- от 17 до 25 (включительно): **“youth”**,
- старше 25: **“adult”**.

После этого скрипт опять выводит сообщение **“enter your name:”** и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **“bye”** и закончить свою работу (выход из цикла!).

```
gkibragimov@fedora:~$ ./test.sh
enter your name:
Gadzhi
enter your age:
25
Gadzhi, your group is youth
enter your name:
Hdazhi
enter your age:
123
Hdazhi, your group is adult
enter your name:
121
enter your age:
s
bye
```

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

```
1 child=16
2 adult=25
3 stdout=0
4
5 while [[ $stdout != 1 ]]
6 do
7     echo "enter your name: "
8     read name
9     if [[ (-z $name) || ($name = 0) ]]; then
10         echo "bye"
11         stdout=1
12     elif [[ -n $name ]]; then
13         while [[ $stdout != 1 ]]; do
14             echo "enter your age: "
15             read age
16             if [[ ($age -eq 0) || (-z $age) ]]; then
17                 echo "bye"
18                 stdout=1
19             elif [[ $age -le $child ]]; then
20                 echo "$name, your group is child"
21             elif [[ $age -gt $adult ]]; then
22                 echo "$name, your group is adult" ;else
23                 if [[ ($age -ge 17) && ($age -le 25) ]]; then
24                     echo "$name, your group is youth" ;fi
25             fi ;break
26         done ;fi
27 done
```

4. Скрипты на bash: разное Какие(ая) из предложенных ниже инструкций увеличат значение переменной a на значение переменной b? Например, если в a было записано 10, в b было 5, то в a должно записаться 15. Выберите **все подходящие** варианты!

```
gkibragimov@fedora:~$ ./test.sh
15
15
15
15
15
gkibragimov@fedora:~$
```

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ let "a=\$a+\$b"
- ☒ let a=a+b
- ☒ let a=\$a+\$b
- ☒ let "a+=b"
- ☒ let "a = a + b"

Пусть вы находитесь в директории /home/bi/Documents/ и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
```

```
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда echo на экран?

Выберите один вариант из списка

✔ Здорово, всё верно.

- ☐ /home/bi/Documents
- ☐ `pwd`
- ☒ /home/bi
- ☐ pwd
- ☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☒ `if `program > some_file.txt``
- ☐ `if [[ `program` -eq 0 ]]`
- ☐ Сначала `var="program"`, затем `if [[ $var -eq 0 ]]`
- ☐ Ничего сделать нельзя

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: `functions1.sh`, `functions2.sh`.

Посмотрите на функцию из `bash`-скрипта:

```
counter () # takes one argument
{
    local let "c1+=$1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже строку, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте после десяти вызовов функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое). В этом случае имеет смысл написать небольшой скрипт на `bash`, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Воспользуемся подсказкой и посчитаем в уме:

Напишите текст

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

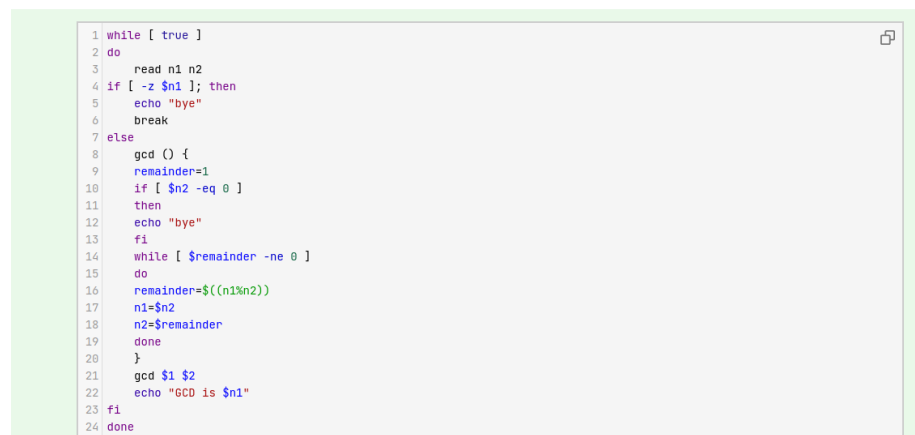
Напишите скрипт на bash, который будет искать наибольший общий делитель (НОД, greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение **“GCD is ”**, например, для чисел 15 и 25 это будет **“GCD is 5”**. После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран **“bye”** и закончить свою работу.

Вычисление НОД несложно реализовать с помощью алгоритма Евклида. Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **М** и **N**). Если аргументы равны, то мы нашли НОД – он равен **М** (или **N**), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **М больше N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем **(М-N)**, а в качестве второго **N**. Если же наоборот, **М меньше N**, то запускаем функцию `gcd` с первым аргументом **М**, а вторым **(N-M)**.

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1

bye
```

A screenshot of a terminal window with a light green border. The terminal displays a bash script for calculating the GCD using the Euclidean algorithm. The script is as follows:

```
1 while [ true ]
2 do
3   read n1 n2
4   if [ -z $n1 ]; then
5     echo "bye"
6     break
7   else
8     gcd () {
9       remainder=1
10      if [ $n2 -eq 0 ]
11      then
12        echo "bye"
13      fi
14      while [ $remainder -ne 0 ]
15      do
16        remainder=$((n1%n2))
17        n1=$n2
18        n2=$remainder
19      done
20    }
21    gcd $1 $2
22    echo "GCD is $n1"
23  fi
24 done
```

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **“exit”**. В этом случае скрипт должен вывести на экран слово **“bye”** и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из **“+”, “-”, “*”, “/”, “%”, “**”**) и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **“error”** и завершить работу.

```
1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ "^[0-9]+$" && "$ikkinchi" =~ "^[0-9]+$" ]]
10    then
11        echo "error"
12        break
13    else
14        case $amal in
15            "+") let "result = birinchi + ikkinchi";;
16            "-") let "result = birinchi - ikkinchi";;
17            "/" let "result = birinchi / ikkinchi";;
18            "*" let "result = birinchi * ikkinchi";;
19            "%" let "result = birinchi % ikkinchi";;
20            "**") let "result = birinchi ** ikkinchi";;
21            *) echo "error" ; break ;;
22        esac
23        echo "$result"
24    fi
25 done
```

5. Продвинутый поиск и редактирование Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"`?

Выберите все подходящие ответы из списка

✓ Верно.

- ☐ `star_trek OST.mp3`
- ☐ `stardust.mpeg`
- ☐ `Eddard_Stark_biography.txt`
- ☒ `STARS.txt`
- ☒ `Star_Wars.avi`

Следующий шаг

Решить снова

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же

Следующий шаг

Решить снова

На самом нижнем уровне не будет поиска

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☐ Все три файла
- ☐ Только `file3`
- ☒ Все кроме `file3`
- ☐ Все кроме `file1`

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в **каждой строке** есть слово `'word'`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -C 1 "word" file.txt > results.txt`

Следующий шаг

Решить снова

Обратим внимание на конец команды (buntu)

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☐ Lubuntu is better than Windows
- ☐ Uuuubuntu!
- ☐ Well, xubuntu is OK
- ☒ Hmm, XKLubuntu
- ☒ The best OS is Xubuntu
- ☒ Lubuntu is better than Ubuntu

Следующий шаг

Решить снова

По умолчанию вывод дважды

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

☒ Хорошие новости, верно!

- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ Появится сообщение об ошибке
- ☐ На экран ничего не напечатается
- ☒ Каждая строчка будет выведена два раза

Следующий шаг

Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 29 965 учащихся
Из всех попыток 40% верных

Запишите в форму ниже инструкцию `sed`, которая заменит все “*аббревиатуры*” в файле `input.txt` на слово “*abbreviation*” и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в **инструкции должны быть** указаны и сам `sed`, и оба файла!

Под “*аббревиатурой*” будем понимать слово, которое удовлетворяет следующим

условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

Напишите текст

✓ Верно.

```
sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Вывод

В ходе выполнения данной лабораторной работы я ознакомился с продвинутыми скриптами консоли ОС Linux